



Institute of
Information Technology

CYBERSECURITY LABORATORY

Lab Work 02

Network Reconnaissance, Vulnerability Assessment, and Controlled Exploitation

Nmap · NSE · OPENVAS / GVM · Metasploit Framework

Course: Intrusion Detection System and Intrusion Prevention System

All Slide Shows: <https://cygnussec.com/project/ids-ips>



Prerequisites

Students should be familiar with:

- Virtualization platforms such as VMware Workstation, VMware ESXi, VirtualBox, or Hyper-V
- Basic TCP/IP networking concepts
- Linux fundamentals and command-line operations
- Basic concepts of ARP, ICMP, TCP, UDP, ports, and services



Learning Objectives

By the end of this lab, students will be able to:

- Explain the major phases of a network security assessment
- Perform host discovery and port scanning using Nmap
- Detect services, software versions, and operating systems



Learning Objectives (continued)

- Use Nmap NSE and OPENVAS / GVM for vulnerability checks
- Interpret findings, CVSS scores, and detection confidence
- Perform controlled exploitation on an approved finding



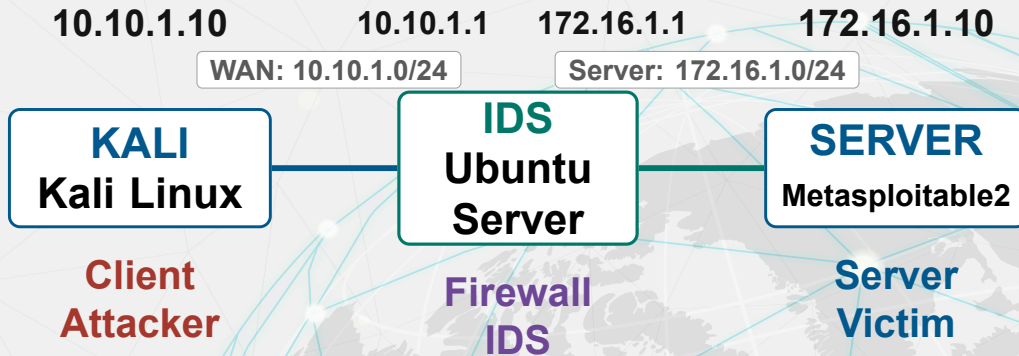
Authorization and Lab Safety

- Verify the target IP and use only assigned, intentionally vulnerable lab systems
- Stop after proof of access; preserve evidence for the report

**Only test systems for which
you have explicit authorization.**

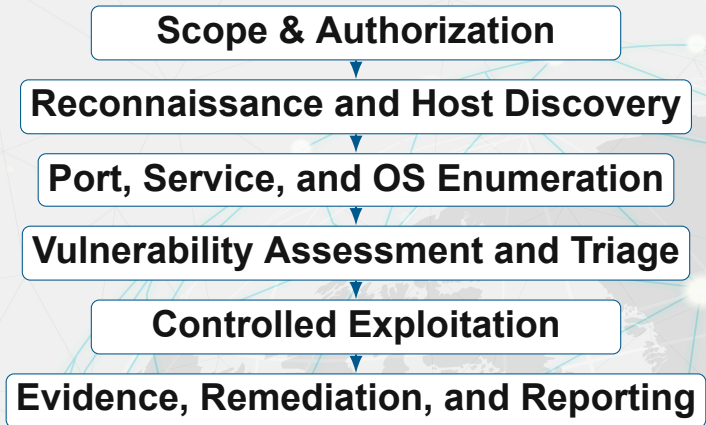


Authorized Lab Environment





Security Assessment Workflow





Lab Outline

1. Security Assessment Overview
2. Network Reconnaissance
3. Host Discovery
4. Port Scanning
5. Service and OS Enumeration



Lab Outline (continued)

6. Vulnerability Detection with Nmap NSE
7. Vulnerability Assessment with OPENVAS / GVM
8. Vulnerability Analysis and Triage
9. Controlled Exploitation with Metasploit Framework
10. Evidence Collection, Remediation, and Reporting



Scanning Types.

- Computer is alive?
- Which ports are open?
- Which services and OS the computer are running?
- Which vulnerabilities may exist?



Scanning Tools.

- Many scanning tools exist today.
- Example: nmap, open-vas, nexpose, nessus and wireless scan, burp suite and so on.
- nmap: scanning types 1 - 3
- OpenVas-greenbone: scanning type 4



nmap (Network Mapper).

- Nmap provides host discovery, port scanning, service detection, and OS detection.
- To accomplish the above, nmap sends specially crafted packets to targets and then analyzes the responses.
- Nmap is available on Linux, Windows, and macOS.
- Install on Linux: `sudo apt install nmap`



Networking Basics For Using nmap.

- Address Resolution Protocol (ARP) Basics
- Internet Protocol (IP) Basics
- Internet Control Message Protocol (ICMP) Basics
- Transport Control Protocol (TCP) Basics



How to use nmap.

- General tips for using nmap
- Basic Command
- Host Discovery
- Port scanning
- Service Detection



How to use nmap. (continued)

- OS Detection
- Output options
- The nmap official manual is your best reference:
<https://nmap.org/book/man.html>



nmap: General tip 1.

- Observation: nmap has many options for customising its behaviour. You cannot remember all of them.
- Tip: nmap has default behaviour if no options are specified, so you should remember its default behaviour, which is easy.
- If you want to modify the default behaviour, apply an option.



nmap: General tip 2.

Nmap performs discovery and enumeration in a clear sequence, making each stage easier to interpret.

Host Discovery

Port Scanning

**Service and OS
Detection**



nmap: General tip 3.

- If you forget about an option, the easiest way is to run 'nmap -h' or look at the "Options Summary" section of 'man nmap'.
- Nmap options are case-sensitive.
- Important options:



nmap: General tip 3. (continued)

- -P: means Ping, controlling host discovery
- -p: means port, specifying port range
- -s: means scan, controlling port and service scanning behaviour.
- -O: means OS, controlling OS detection



nmap: General tip 3. (continued)

- -o: means output, specifying output format
- -A: means Aggressive or All, doing all of port scanning, service detection and OS detection.



nmap: Basic Command.

- `└─(kali@kali)-[~]`
- `sudo nmap 172.16.1.10`
- `[sudo]` password for kali:
- Starting nmap 7.94SVN (<https://nmap.org>) at 2023-09-22 09:16 CDT



nmap: Basic Command. (continued)

- `nmap scan report for 172.16.1.10`
- Host is up (0.00014s latency).
- Not shown: 977 closed tcp ports (reset)
- PORT STATE SERVICE
- 21/tcp open ftp



nmap: Basic Command. (continued)

- 22/tcp open ssh
- 23/tcp open telnet
- 25/tcp open smtp
- 53/tcp open domain
- 80/tcp open http



nmap: Basic Command. (continued)

- 111/tcp open rpcbind
- 139/tcp open netbios-ssn
- nmap done: 1 IP address (1 host up) scanned in 0.17 seconds



nmap: Host Discovery - A Range of IPs.

- A range of IP addresses can be specified with –:
- `sudo nmap 172.16.1.2-64`
- Scan all hosts on a subnet
- `sudo nmap 172.16.1.*`
- `sudo nmap 172.16.1.0/24`



nmap: Host Discovery - Options.

- Nmap allows you to specify host discovery options (also called "Ping Options") by starting with "-P".
- Skip host discovery completely (-PN or -Pn)
- `sudo nmap -Pn 172.16.1.10`
- Only do ICMP Echo Request (-PE)



nmap: Host Discovery - Options. (continued)

- `sudo nmap -PE 172.16.1.10`
- Only do ICMP Timestamp Request (-PP)
- `sudo nmap -PP 172.16.1.10`



nmap: Host Discovery - Options.

- Only do TCP SYN (-PS) : port 80 will be probed by default; can change by -PS<port number>.
- `sudo nmap -PS 172.16.1.10 # using port 80`
- `sudo nmap -PS443 172.16.1.10 # using port 443`



nmap: Host Discovery - Options. (continued)

- Only do TCP ACK (-PA) : port 80 will be probed by default; can change by -PA<port number>.
- `sudo nmap -PA 172.16.1.10 # using port 80`
- `sudo nmap -PA443 172.16.1.10 # using port 443`



nmap: Host Discovery - Options. (continued)

- Only do UDP (-PU) : port 40125 will be probed by default; can change by -PU<port number>.
- `sudo nmap -PU 172.16.1.10 # using port 40125`
- `sudo nmap -PU56667 172.16.1.10 # using port 56667`



nmap: Port Scanning - Six Port States.

Nmap reports one of six states for each scanned port:

- **open**: An application accepts TCP connections, UDP datagrams, or SCTP associations on the port.
- **closed**: The host responds to probes, but no application is listening on the port.
- **filtered**: Packet filtering prevents the probes from reaching the port, so Nmap cannot determine whether it is open.



nmap: Port Scanning - Six Port States. (continued)

Three states indicate an ambiguous scan result:

- **unfiltered**: The port is accessible, but Nmap cannot tell whether it is open or closed.
- **open|filtered**: Nmap cannot tell whether the port is open or filtered.
- **closed|filtered**: Nmap cannot tell whether the port is closed or filtered; this state is used only by the IP ID idle scan.



nmap: Port Scanning - Options.

- To change the default behaviour, nmap allows you to specify scanning options by starting with "-s".
- Scan ports by SYN packets (-sS)
- `sudo nmap -sS 172.16.1.10`
- Scan ports by full TCP connections (-sT)



nmap: Port Scanning - Options. (continued)

- `sudo nmap -sT 172.16.1.10`
- Scan ports by ACK packets (-sA)
- `sudo nmap -sA 172.16.1.10`



nmap: Port Scanning - Options.

- Scan ports by FIN packets (-sF)
- `sudo nmap -sF 172.16.1.10`
- Scan ports by turning on URG, FIN and PSH (-sX, so called Xmas Scan)



nmap: Port Scanning - Options. (continued)

- light up a probe packet like a Xmas tree; get OS or firewall info by observing how they react to such a packet
- `sudo nmap -sX 172.16.1.10`
- Scan the basic 1000 UDP ports (-sU)
- `sudo nmap -sU 172.16.1.10`



nmap: Port Scanning - Options. (continued)

- Not scan any port, only do host discovery (-sn)
- `sudo nmap -sn 172.16.1.10`



nmap: Port Scanning - Specify Ports.

- Other than the default 1000 ports, nmap allows you to specify ports by the "-p" option.
- Scan port 80 only
- `sudo nmap -p 80 172.16.1.10`
- Scan ports 20-28,80, and 100
- `sudo nmap -p 20-28,80,100 172.16.1.10`



nmap: Port Scanning - Specify Ports.

- The "--top-ports" option is used to scan any number of top ranked ports.
- Scan the top 100 ports
- `sudo nmap --top-ports 100 172.16.1.10`
- Command to show the top 100 TCP ports:



nmap: Port Scanning - Specify Ports. (continued)

- `sudo nmap -sT --top-ports 100 -v -oG -`
- Command to show the top 100 UDP ports:
- `sudo nmap -sU --top-ports 100 -v -oG -`



nmap: Service detection and OS detection - Option.

- **-O (Enable OS detection)** Enables OS detection, as discussed above. Alternatively, you can use **-A** to enable OS detection along with other things.
- **-osscan-limit** (Limit OS detection to promising targets)



nmap: Service detection.

- Nmap maintains a database file called `nmap-service-probes`, which contains:
- probes for detecting various services.
- fingerprints to match responses to a certain service.
- It is located at `/usr/share/nmap/nmap-service-probes`



nmap: Service Detection - How It Works and Options.

- Using the `nmap-service-probes` database, Nmap identifies the service protocol, application, and version running on each port.
- Examples include FTP, SSH, Telnet, and HTTP services such as `vsftpd` or `Apache httpd`.
- Enable service detection with `-sV`:
- `sudo nmap -sV 172.16.1.10`



nmap: Service Detection - Options. (continued)

- Detecting services on UDP ports at the host 172.16.1.10 (service detection on UDP ports takes very long time)
- `sudo nmap -sU -sV 172.16.1.10`
- `sudo nmap -sU -sV -p U:53 172.16.1.10`



nmap: Service Detection - Output.

- `└─(kali@kali)-[~]`
- `sudo nmap -sV 172.16.1.10`
- Starting Nmap 7.94SVN (<https://nmap.org>) at 2023-09-22 10:42 CDT
- `Nmap scan report for 172.16.1.10`



nmap: Service Detection - Output. **(continued)**

- Host is up (0.00020s latency).
- Not shown: 977 closed tcp ports (reset)
- PORT STATE SERVICE VERSION
- 21/tcp open ftp vsftpd 2.3.4



nmap: Service Detection - Output. **(continued)**

- 22/tcp open ssh OpenSSH 4.7p1 Debian 8ubuntu1 (protocol 2.0)
- 23/tcp open telnet Linux telnetd
- 25/tcp open smtp Postfix smtpd
- 53/tcp open domain ISC BIND 9.4.2



nmap: Service Detection - Output. **(continued)**

- 80/tcp open http Apache httpd 2.2.8 ((Ubuntu) DAV/2)
- 111/tcp open rpcbind 2 (RPC #100000)
- 139/tcp open netbios-ssn Samba smbd 3.X - 4.X
(workgroup: WORKGROUP)



nmap: Service Detection - Output. (continued)

- 445/tcp open netbios-ssn Samba smbd 3.X - 4.X
(workgroup: WORKGROUP)
- ...



nmap: Service Detection - Output. (continued)

- Service Info: Hosts: metasploitable.localdomain, irc.Metasploitable.LAN; OSs: Unix, Linux; CPE: cpe:/o:linux:linux_kernel
- Service detection performed. Please report any incorrect results at <https://nmap.org/submit/>.
- Nmap done: 1 IP address (1 host up) scanned in 52.37 seconds



nmap: Operating System Detection.

- Nmap maintains the `nmap-os-db` database, which contains:
- probes for detecting various OSes.
- fingerprints to match responses to a certain OS.
- It is located at `/usr/share/nmap/nmap-os-db`
- Based on this database, nmap tries to determine the OS of a target.



nmap: OS Detection - Options.

- To enable OS detection, simply use the argument "-O".
- Detecting the OS of host 172.16.1.10
- `sudo nmap -O 172.16.1.10`



nmap: OS Detection - Output.

- `└─(kali@kali)-[~]`
- `sudo nmap -O 172.16.1.10`
- Starting Nmap 7.94SVN (<https://nmap.org>) at 2023-09-22 10:47 CDT
- `Nmap scan report for 172.16.1.10`



nmap: OS Detection - Output. **(continued)**

- Host is up (0.00015s latency).
- Not shown: 977 closed tcp ports (reset)
- PORT STATE SERVICE
- ...
- Device type: general purpose



nmap: OS Detection - Output. (continued)

- Running: Linux 2.6.X
- OS CPE: `cpe:/o:linux:linux_kernel:2.6`
- OS details: Linux 2.6.9 - 2.6.33
- Network Distance: 2 hops



nmap: OS Detection - Output. (continued)

- OS detection performed. Please report any incorrect results at <https://nmap.org/submit/> .
- Nmap done: 1 IP address (1 host up) scanned in 2.48 seconds



nmap: OS Detection - Output Notes.

- "-O" is different from "-sV" in:
- No service version info
- More detailed OS info
- Much quicker to complete



nmap: Output Formats.

- A scanning tool is as useful as the output it generates.
- Besides outputting to the screen, nmap can generate other output formats.
- These are for the convenience of being imported and used by other programs such as network analysis tools, exploitation tools, etc.



nmap: Five Output Formats By nmap.

- Interactive output (the default): output to the screen.
- Normal output: write the output to a text file. The content is the same as the interactive output except that it contains less runtime information
- XML output: write the output to an XML file



nmap: Five Output Formats By nmap. (continued)

- Very useful for being imported to other programs.
- Grepable output: write the output to a text file in the format easy for the 'grep' tool.
- Kiddie output: this output format is for fun. You can ignore it.



nmap: Output formats - Options.

- Interactive output (the default): no argument is needed; always accompany other output formats.
- Normal output: `nmap -oN target.nmap 172.16.1.10`
- XML output: `nmap -oX target.xml 172.16.1.10`
- Grepable output: `nmap -oG target.gnmap
172.16.1.10`



nmap: Output formats - Options. (continued)

- All three outputs: `nmap -oA target 172.16.1.10`
- Will generate three files: `target.nmap`, `target.xml` and `target.gnmap`



nmap: Scripting Engine (NSE).

- NSE allows users to write (and share) simple scripts to automate a wide variety of networking tasks.
- Tasks typically include:
- Customized network/host discovery
- More sophisticated version detection



nmap: Scripting Engine (NSE). **(continued)**

- Vulnerability detection
- This unit will not require you to write scripts for NSE, but you will be required to use some existing NSE scripts later.
- Located at `/usr/share/nmap/scripts`



nmap: Example: Running an NSE script.

- The script name
- The port number
- The target IP
- Vuln confirmed

```
(kali@kali) ~$ sudo nmap --script=ftp-vsftpd-backdoor -p 21 172.16.1.18
Starting Nmap 7.942WN ( https://nmap.org ) at 2024-09-22 10:53 CDT
Nmap scan report for 172.16.1.18
Host is up (0.0003s latency).

PORT      STATE SERVICE
21/tcp    open  ftp
| ftp-vsftpd-backdoor:
| | VULNERABLE:
| |   vsFTPd version 2.3.4 backdoor
| |   State: VULNERABLE (Exploitable)
| |   ID: CVE-2011-2523 BID:48539
| |   vsFTPd version 2.3.4 backdoor, this was reported on 2011-07-04.
| |   Disclosure date: 2011-07-03
| |   Exploit results:
| |     Shell command: id
| |     Results: uid=0(root) gid=0(root)
| |   References:
| |     http://scarybeastsecurity.blogspot.com/2011/07/alert-vsftpd-download-backdoored.html
| |     https://cve.mitre.org/cgi-bin/cvename.cgi?name=CVE-2011-2523
| |     https://github.com/rapid7/metasploit-framework/blob/master/modules/exploits/unix/ftp/vsftpd_234_backdoor.rb
| |     https://www.securityfocus.com/bid/48539
|_
Nmap done: 1 IP address (1 host up) scanned in 1.19 seconds
```



nmap: NSE Vulnerability Scripts.

- The scripts which come under the "vuln" category look for specific known vulnerabilities and only report back if any are identified in the target system.
- `sudo nmap -sV --script vuln 172.16.1.10`



GVM (Greenbone Vulnerability Management).

- A fork of Nessus in 2005 when the open source Nessus was acquired by Tenable Network Security and became proprietary.
- GVM is currently maintained by Greenbone Networks.
- Official website: www.openvas.org



GVM (Greenbone Vulnerability Management). (continued)

- OpenVAS is the scanner component of the GVM framework.
- OpenVAS stands for Open Vulnerability Assessment Scanner.
- According to its website, it is currently used by many pentesting companies and government organizations.



GVM - Working Principle.

- GVM maintains a public feed of Network Vulnerability Tests (NVTs).
- The feed now contains more than 80,000 NVTs.
- Each NVT is a NASL script for detecting a certain vulnerability.



GVM - Working Principle. (continued)

- NASL (Nessus Attack Scripting Language): a scripting language from Nessus dedicated to vuln detection.
- GVM executes these NVTs to detect vulns.



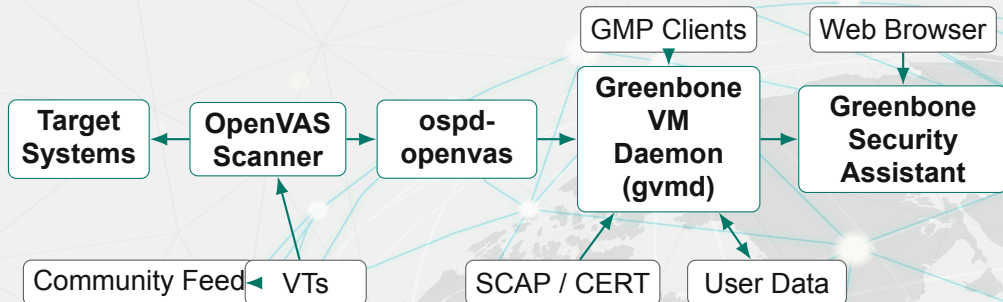
GVM - Architecture.

- GVM mainly consists of three components:
- Greenbone Security Assistant (GSA)
- Greenbone Vulnerability Manager (GVM)
- OpenVAS Scanner
- They are started with:
- `sudo gvm-start`

GVM - Architecture.



Institute of
Information Technology





Greenbone Security Assistant (GSA).

- Providing a web interface for GVM:
- User login
- Start/stop scanner
- Scanner Configurations, etc.
- It's a daemon (named gsad) listening at port 9392



Greenbone Security Assistant (GSA). (continued)

- Contains a built-in light-weight web server (microhttpd)
- Accessible by HTTPS at: `https://localhost:9392`
- It also listens on port 80 with HTTP. If you visit `http://localhost:80/`, you'll be redirected to `https://localhost:9392/`



GVM - Manager.

- Main functions:
- Schedules the scanner
- Maintains a database which stores all configuration data and scan results
- Generates scan results in different formats: pdf, txt, xml, html, latex, csv, etc.
- Manages user accounts



GVM - Manager. (continued)

- A daemon (called `gvmd`) listening on a system port (not a TCP port by default), and receiving commands from Greenbone Security Assistant



GVM - OpenVAS Scanner.

- Controlled by GVM Manager
- Main functions:
- Scan multiple targets concurrently 0 Executes the NVTs from the public GVM NVT Feed
- Return the results to GVM Manager



GVM - Installation Steps.

- You should first get information on the newest versions of packages and their dependencies for your Kali:
- `└─(kali@kali)-[~]`
- `sudo apt update`



GVM - Installation Steps. (continued)

- - "apt" is a package management tool for Debian-derived Linux distros.
- - "apt update" only gets information, and will not install the updates for you.



GVM - Installation Steps. (continued)

- - You may have heard of "apt-get" instead before, but it is an older tool. For the difference between "apt" and "apt-get", see:
<https://itsfoss.com/apt-vs-apt-get-difference/>



GVM - Installation Steps.

- Install GVM:
- `└─(kali@kali)-[~]`
- `sudo apt install gvm`
- NB: The trailing star means all packages starting with 'gvm'.



GVM - Installation Steps. (continued)

- Setup GVM:
- `└─(kali@kali)-[~]`
- `sudo gvm-setup`
- This step will take a long time due to the download of latest CVE info and Network Vulnerability Tests (NVT) scripts.



GVM - Installation Steps.

- Set up the username and password for accessing GVM:
- `$ sudo runuser -u _gvm --gvmd --user=admin --new-password=letmein`
- NB: The 'gvmd' command needs to be run under the user '_gvm', which explains why 'runuser -u _gvm --'.



GVM - Installation Steps. (continued)

- The username (admin) and password (letmein) above will be used in the Greenbone Security Assistant web interface.
- The password is of your choice.



GVM - Launch.

- To launch GVM, you need to run:
- `└─(kali@kali)-[~]`
- `sudo gvm-start`
- NB: This may take 1-2 minutes. Please be patient.



GVM - Launch. (continued)

- At the end of this script, it will try to automatically launch Firefox to access GVM at <https://localhost:9392>
- This may not always succeed. If so, you have to open Firefox and visit this link yourself.
- At Firefox, accept the untrusted certificate and choose to proceed to this link.



GVM - Launch.

- Enter the username and password set up in the previous slide:

A screenshot of the Greenbone login interface. It features a green border around a white central area. At the top left is the Greenbone logo (a green dragon head) and the text "Greenbone". Below this is the heading "Sign in to your account". There are two input fields: "Username" with the value "admin" and "Password" with masked characters "*****". A dark grey "Sign In" button is positioned below the password field. At the bottom center is a green hexagonal badge with the text "Greenbone Community" and "EDITION" in a black diamond shape.



GVM - Launch.

- Check whether GVM launched successfully:
- - That is, the Greenbone Security Assistant daemon (gsad) should be listening on port 9392 with HTTPS protocol.

```
(kali@kali) ~  
$ sudo ss -ltnp  
State Recv-Q Send-Q Local Address:Port Peer Address:Port Process  
LISTEN 0 244 127.0.0.1:5432 0.0.0.0:* users:({"postgres",pid=5987,  
LISTEN 0 4096 127.0.0.1:9392 0.0.0.0:* users:({"gsad",pid=25645,fd=  
LISTEN 0 4096 127.0.0.1:80 0.0.0.0:* users:({"gsad",pid=25646,fd=  
LISTEN 0 244 [::]:5432 [::]:* users:({"postgres",pid=5987,
```



GVM - Launch. (continued)

- NB: it also listens on port 80 with HTTP protocol. If you visit <http://localhost:80/>, you'll be redirected to <https://localhost:9392/>



GVM - Launch

- Check whether GVM launched successfully:
- The Greenbone Security Assistant daemon (gsad) should listen on HTTPS port **9392**.
- HTTP port **80** redirects to `https://localhost:9392/`.

```
(kali@kali) - [~]  
$ sudo ss -lntp
```

State	Recv-Q	Send-Q	Local Address:Port	Peer Address:Port	Process
LISTEN	0	244	127.0.0.1:5432	0.0.0.0:*	users:(("postgres",pid=5907,
LISTEN	0	4096	127.0.0.1:9392	0.0.0.0:*	users:(("gsad",pid=25645,fd=
LISTEN	0	4096	127.0.0.1:80	0.0.0.0:*	users:(("gsad",pid=25646,fd=
LISTEN	0	244	:::1:5432	:::.*	users:(("postgres",pid=5907,



GVM - Stop.

- If you want to shutdown GVM, you should do:
- `└─(kali@kali)-[~]`
- `sudo gvm-stop`



GVM - Usage.

- Create a target
- Create a task
- Start the task
- Create a GSA user
- Other menu items
- To perform a vuln scan on a target, these three steps are typically needed.



GVM - Create A Target.

- Go to the "Configuration" menu and click "Targets"
- Click the "*" icon on the top to create a new target.
- Follow the screenshot in the next slide to do the following:
- Enter a name for this target



GVM - Create A Target. (continued)

- Enter the IP address of the host (typically, a target consists of one host)
- Select the Port List to scan: use the default
- Select the Alive Test (i.e., Host Discovery) method: use the default



Creating A Target Screenshot.

New Target

Name

Meta

Comment

Hosts

☒ Manual

172.16.1.10

☐ From file

Choose File

No file chosen

Exclude Hosts

☒ Manual

☐ From file

Choose File

No file chosen

Allow simultaneous scanning via multiple IPs

☒ Yes

☐ No

Port List

All IANA assigned TCP

*

Alive Test

Scan Config Default

Credentials for authenticated checks

SSH

--

on port

22

*

SMB

--

*

Cancel

Save



GVM - Notes For Creating A Target.

- Similar to nmap, GVM also has default behaviour for:
- Which ports to scan
- How to discover alive hosts



GVM - Notes For Creating A Target. (continued)

- You can simply choose the default behaviour in the previous screenshot if you don't have any particular requirements.
- You can Click 'Configuration' -> 'Port Lists' to see the details of each port list option.



GVM - Create A Task.

- Follow the screenshot in the next two slides to do the following:
- Enter a Name for this task
- Select the Scan Target
- Select the Scan Config: typically use 'Full and Fast'.



GVM - Create A Task. (continued)

- Click 'Configuration' -> 'Scan Configs' to see what types of scans are included in each Scan Config.



GVM - Creating a Task: Quality of Detection

- Note the Quality of Detection (QoD) here. You can use the default setting of 70%.
- Scan results with less than 70% QoD will not be reported.

The screenshot shows the 'New Task' dialog box in GVM. The 'Min QoD' field is set to 70%. The 'Auto Delete Reports' section is also visible, with options for 'Do not automatically delete reports' (selected) and 'Automatically delete oldest reports but always keep newest' (unselected). The 'Scanner' is set to 'OpenVAS Default' and the 'Scan Config' is set to 'Full and fast'.



GVM - Start The Task

- Click the **Start** icon to begin the task.
- The web interface automatically refreshes to show scan progress.
- Refresh the page manually if the progress is not updated quickly enough.

Meta	Done	12 Oct 2023 · 12:17 AM	10.0 (High)	Start	Edit	Report
Meta2	New			Start	Edit	Report



GVM - Other Menu Items: SecInfo.

- Within it, click "NVTs" to see info on all NVTs, etc.
- Within it, click "CVEs" to see info on all known CVEs, etc.
- Within it, click "CPEs" to see info on all known CPEs, etc.



GVM - Other Menu Items: SecInfo. (continued)

- Therefore, GVM includes a lot of very useful information.

GVM - Other Menu Items: SecInfo.



Institute of
Information Technology

**Report:Thu, Oct 12, 2023 12:17 AM UTC**

Done

ID: 134eac98-9846-4e29-a2be-ce59c76d1ea7Created: Thu, Oct 12, 2023 12:17 AM UTCModified:

Information	Results (69 of 587)	Hosts (1 of 1)	Ports (20 of 23)	Applications (16 of 16)	Operating Systems (1 of 1)	CVEs (34 of 34)	Closed CVEs (0 of 0)	TLS Certificates (2 of 2)	Error Messages (0 of 0)	User Tags (0)
-------------	------------------------	-------------------	---------------------	----------------------------	-------------------------------	--------------------	-------------------------	------------------------------	----------------------------	------------------

Task Name

Meta

Scan Time

Thu, Oct 12, 2023 12:17 AM UTC - Thu, Oct 12, 2023 12:45 AM UTC

Scan Duration

0:28 h

Scan Status

Done

Hosts scanned

1

Filter

apply_overrides=0 levels=hml min_qod=70

Timezone

Coordinated Universal Time (UTC)

GVM - Report



Institute of
Information Technology

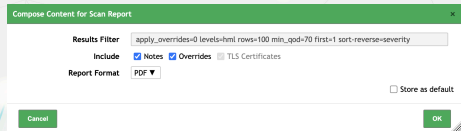
- The report provides a detailed account of the scan results.
- Open it from **Scans** → **Reports**.
- Click the date field to view the report.





GVM - Report - Download.

- Available in many formats such as pdf, xml, latex, txt, etc.
- The conversion to certain file type may take some time.





GVM - Report - Download. (continued)

- Click this dropdown list to select download file type



GVM - PDF Report - Structure.

- The pdf report has four parts:
- Summary
- Table of Contents
- Result Overview
- Results per Host



GVM - PDF Report - Result Overview.

- The 'Result Overview' section is very important, giving a summary on the numbers of results in different severity levels.
- The severity levels are categorized according to CVSS:
- High: 7.0 - 10.0



GVM - PDF Report - Result Overview. (continued)

- Medium: 4.0 - 6.9
- Low: 0.1 - 3.9
- Log: 0.0



GVM - PDF Report - A Result.

- A result in GVM report means a positive outcome from a NVT.
- A result has the following fields (see the screenshot in next slide):
- Vulnerability or NVT name



GVM - PDF Report - A Result. (continued)

- Severity: the CVSS score
- Solution Type: Vendor Fix, Mitigation, or Workaround
- QoD (Quality of Detection): how confident it is positive, in percentage
- Host: the IP address of the scanned host
- Location: the port number



GVM - PDF Report - An Exemplar Result.

High (CVSS: 7.5) NVT: Java RMI Server Insecure Default Configuration RCE Vulnerability
Summary Multiple Java products that implement the RMI Server contain a vulnerability that could allow an unauthenticated, remote attacker to execute arbitrary code (remote code execution/RCE) on a targeted system with elevated privileges.
Vulnerability Detection Result By doing an RMI request it was possible to trigger the vulnerability and make the remote host send a request back to the scanner host (Details on the received packet follows). Destination IP: 192.168.1.85 (receiving IP on scanner host side) Destination port: 28495/tcp (receiving port on scanner host side) Originating IP: 192.168.1.91 (originating IP from target host side)
Impact An unauthenticated, remote attacker could exploit the vulnerability by transmitting crafted packets to the affected software. When the packets are processed, the attacker could execute arbitrary code on the system with elevated privileges.
Solution: Solution type: Workaround Disable class-loading. Please contact the vendor of the affected system for additional guidance.
Vulnerability Insight The vulnerability exists because of an incorrect default configuration of the Remote Method Invocation (RMI) Server in the affected software.



GVM - PDF Report - Results per Host.

- The 'Result per Host' section gives a detailed account of each result.
- The results are first classified according to hosts.
- In our cases, we scan one host only, so we only see one subsection for the scanned host here.



GVM - PDF Report - Results per Host. (continued)

- The results are then classified according to ports.
- There can be more than one results under a port.



Exploitation Using Metasploit-Framework.

- 1: Reconnaissance
- 2: Scanning
- 3: Exploitation
- 4: Post Exploitation
- 5: Writing Report



Exploitation Basics.

- It is after a vuln is discovered on a target
- It basically involves two steps:
- Run a piece of code to gain the access to the target by exploiting the vuln.
- This piece of code is typically called an exploit.



Exploitation Basics. (continued)

- Run another piece of code to control the target, such as a shell.
- This piece of code is typically called a payload or shellcode.



Exploitation Tools.

- Metasploit Framework (MSF)
- Free Open Source Software.
- The most popular one.
- An important topic in this subject.
- Metasploit Pro



Exploitation Tools. (continued)

- Commercial version, by Rapid7 Ltd.
- Core Impact
- Another commercial software.
- Effective but less popular than Metasploit.
- And more... or you can develop exploits by yourself.



An Important Exploit Database.

- The most popular open source database for known exploits is exploit-db.
- Maintained by Offensive Security.
- Website: www.exploit-db.com.
- May not be accessible within our university network.



An Important Exploit Database. (continued)

- Git repository:
<https://gitlab.com/exploit-database/exploitdb>.
- Accessible within our university network
- Source codes in various languages such as C, Python, Ruby, Java, etc.



An Important Exploit Database.

- If you want to conduct exploitation without using a tool such as MSF, you can download an exploit from exploit databases and manually apply it.
- For the convenience of accessing exploit-db, Kali has a local copy of it at `/usr/share/exploitdb`
- Kali also provides a tool to search this local copy of exploit-db called `searchsploit`.



searchsploit: Tool To Search exploit-db.

- Usage:
- `searchsploit [options] term1 [term2] ... [termN]`
- You need to give at least one term and can give as many terms as you want.
- Main Options:



searchsploit: Tool To Search exploit-db. (continued)

- -c: Perform a case-sensitive search (Default is case insensitive).
- -h: Show this help screen.
- -p: Show the full path to an exploit.
- -t: Search just the exploit title (Default is title AND the exploit's path).
- -u: Check and install exploitdb package updates.



Searchsploit: Example For Using.

- Example:

```
(kali㉿kali)-[/usr/share/metasploit-framework/modules]
$ searchsploit vsftpd
```

Exploit Title	Path
vsftpd 2.0.5 - 'CmD' (Authenticated) Remote Memory Consumpti	linux/dos/5814.pl
vsftpd 2.0.5 - 'deny file' Option Remote Denial of Service (	windows/dos/31818.sh
vsftpd 2.0.5 - 'deny file' Option Remote Denial of Service (	windows/dos/31819.pl
vsftpd 2.3.2 - Denial of Service	linux/dos/16270.c
vsftpd 2.3.4 - Backdoor Command Execution (Metasploit)	unix/remote/17491.rb

```
Shellcodes: No Results
```



MSF - History.

- An open source software developed by HD Moore in 2003.
- It generated great excitement at that time.
- It implements the common techniques needed in exploitation and post exploitation, and allows plugins from third parties.



MSF - History. (continued)

- Acquired by Rapid7 LLC in October, 2009.
- Since its acquisition, Rapid7 has developed a proprietary edition called Metasploit Pro, while the basic part, still called Metasploit Framework, remains open source.



MSF - Architecture.

- The MSF architecture can be understood more easily by examining its file system.
- The framework is installed under:
- `/usr/share/metasploit-framework/`



MSF - Architecture - Directories.

- The following directories are important:
- data: data files used by Metasploit, e.g., the word list for password dictionary attack
- lib: the core of the framework code



MSF - Architecture - Directories. (continued)

- modules: the arsenal of MSF, encompassing exploits, payloads, scanners, etc.
- plugins: contributed from other parties
- We will focus on modules.



MSF - Modules.

- The main module categories include:
- **exploit**: exploit development.
- **payload**: payload delivery.
- **evasion**: modules that help evade antivirus software.



MSF - Modules. (continued)

- **encoder**: helps upload the payload to the target correctly.
- **nop**: no-operation instructions used to keep payload sizes as desired.
- **auxiliary**: tools that support exploitation, such as scanners, fuzzers, and server captures.



MSF - Modules. (continued)

- Fuzzers generate user inputs to test an application.
- Server captures imitate a server to gather user credentials, such as FTP or HTTP capture.
- **post**: payloads such as backdoors and privilege escalators used during post exploitation.



MSF - Modules.

- The modules are mapped to the following directory structure:
- `/usr/share/metasploit-framework/modules/`
- auxiliary, encoders, evasion, exploits, nops, payloads, and post



Modules - Exploits.

- Because an exploit can usually be applied to only one type of operating system, MSF organizes exploits by operating system.
- Examples include Linux, Windows, Unix, Android, FreeBSD, Solaris, and others.



Modules - Exploits. (continued)

- Within an operating-system directory, exploits are further classified according to the services provided by that operating system.
- Examples include FTP, HTTP, IMAP, MySQL, PostgreSQL, Samba, SMTP, SSH, and Telnet.



Modules - Exploits.

- Each exploit is a Ruby script with the suffix `.rb`.
- Each exploit calls the basic APIs provided by MSF to implement an exploitation and can be launched by MSF.



Modules - Payloads.

- Payloads are broadly classified into three categories:
- Singles
- Stagers
- Stages



Modules - Payloads - Singles.

- Singles can function alone to complete a task.
- For example, they can add a new user or execute a command.



Modules - Payloads - Stagers and Stages.

- Sometimes a payload is too large to fit in the exploited buffer at the victim, so it cannot work as a single.
- It has to be broken into a stager and a stage.



Modules - Payloads - Stagers and Stages. (continued)

- The stager is typically small and can fit into the exploited buffer. Its execution will upload the stage into the victim's memory.
- The stage is typically large and needs to be specially loaded into the victim's memory.



Modules - Payloads - Stagers and Stages. (continued)

- A mnemonic: stage is big, so stage is the bigger one.
- If a payload needs to be broken, we also say it is to be staged.



Modules - Payloads - Stagers.

- There are typically two kinds of stagers:
- bind: create a listening TCP port at the target and wait for the TCP connection from the attacker machine, and then load the stage.



Modules - Payloads - Stagers. (continued)

- reverse: create a listening TCP port at the attacking machine and wait for the TCP connection from the target, and then load the stage.
- Note that the reverse one is more powerful, as most firewalls won't filter outbound connections.



Modules - Payloads - Stages.

- Examples of stages include:
- Shell: provides a command line terminal of the compromised OS.
- Meterpreter: provides a command line terminal with specialised commands for hacking.



Modules - Payloads - Stages. (continued)

- VNC injection: provides a graphical remote desktop.
- And much more ...



MSF - User Interfaces.

- MSF supports the following user interfaces, each with its strengths and weaknesses:
- Command-Line Interfaces:
- msfcli: simple to use, but not so powerful as msfconsole; good for scripting.



MSF - User Interfaces. (continued)

- msfconsole: interactive, having access to almost every feature of MSF, but not good for scripting.
- GUIs
- Metasploit Pro: the commercial one.
- Armitage: written in Java, an open source one.
- This subject will focus on msfconsole as it is the most powerful and helps with understanding.



Msfconsole - Starting.

- Start the PostgreSQL database daemon, as MSF uses the PostgreSQL as the backend.
- `sudo service postgresql start`
- PostgreSQL is another open source database competing with MySQL



Msfconsole - Starting. (continued)

- Initialise the MSF database named 'msf' in PostgreSQL
- `sudo msfdb init`
- only do it once when msfconsole is to run for the first time
- Launch msfconsole (will take a little while)
- `sudo msfconsole`



Msfconsole - Basic Commands.

- Msfconsole includes many commands; the following lists some basic ones.
- help
- info
- search



Msfconsole - Basic Commands. (continued)

- use
- back
- exit



Commands - Help.

- **help**: list all available commands.
- **help <command name>**: display the usage of a command.

```
msf > help info
Usage: info <module name> [mod2 mod3 ...]

Options:
* The flag '-j' will print the data in json format
* The flag '-d' will show the markdown version with a browser.
  More info, but could be slow.
Queries the supplied module or modules for information. If no
module is given,
show info for the currently active module.
```



Commands - info, check.

- `info <module name>`
provides detailed information about a module, including its description and vulnerability references (CVE, BID, URLs, etc.).

```
msf exploit(ms08_067_netapi) > check  
[*] Verifying vulnerable status... (path: 0x0000005a)  
[*] System is not vulnerable (status: 0x00000000)  
[*] The target is not exploitable.
```



Commands - info, check. (continued)

- `msf > info exploit/windows/smb/ms03_049_netapi`
- The output is very long. You should try it in msfconsole yourself.
- `check` is used in an exploit context to test whether the exploit can be applied without actually applying it.



Commands - search.

- `search <patterns>`: looking for a module by searching the patterns in module name, description, references, etc.
- `search eternalblue windows`

```
msf6 > search eternalblue windows

Matching Modules
=====
#  Name                                                                 Disclosure Date   Rank  Check  Description
--  ---                                                                 -
0  auxiliary/admin/smb/ms17_010_command                               2017-03-14      normal No    MS17-010 EternalRomance/EternalSynergy/EternalChampion SMB Remote Windows Command Execution
1  exploit/windows/smb/ms17_010_eternalblue                           2017-03-14      average Yes   MS17-010 EternalBlue SMB Remote Windows Kernel Pool Corruption
2  exploit/windows/smb/ms17_010_eternalblue_win8                     2017-03-14      average No    MS17-010 EternalBlue SMB Remote Windows Kernel Pool Corruption for Win8+

note: windows Kernel Pool Corruption for Win8+
```



Commands - search. (continued)

- "Rank" indicates the usability of this module. Ranks include: excellent, great, good, normal, average, low, and manual.
- "Check" indicates if the 'check' command is supported.



Commands - search.

- You can also search by using keywords.

```
msf > help search
Usage: search [keywords]

Keywords:
  app      : Modules that are client or server attacks
  author   : Modules written by this author
  bid      : Modules with a matching Bugtraq ID
  cve      : Modules with a matching CVE ID
  edb      : Modules with a matching Exploit-DB ID
  name     : Modules with a matching descriptive name
  platform : Modules affecting this platform
  ref      : Modules with a matching ref
  type     : Modules of a specific type (exploit, auxiliary, or post)

Examples:
  search cve:2009 type:exploit app:client
```



Commands - search.

- In addition to the cve or bid mentioned in the last slide, you can also search by using Microsoft Security Bulletin ID for an exploit.

```
msf > search ms08-067
```

Matching Modules

Name	Disclosure Date	Rank
exploit/windows/smb/ms08_067_netapi	2008-10-28	great



Commands - search. (continued)

- E.g., MS08-067: <https://docs.microsoft.com/en-us/security-updates/SecurityBulletins/2008/ms08-067>.



Commands - use.

- `use <module name>` selects a module and enters that module's context.

```
msf > use exploit/unix/ftp/vsftpd_234_backdoor  
msf exploit(vsftpd_234_backdoor) >
```



Commands - use. (continued)

- Under a module context, you can issue commands related to that module. We'll come back to this topic later.



Commands - sessions.

- `use <module name>` selects a module and enters that module's context.
- `sessions -i <session ID>` enters the session with that ID.

```
msf6 exploit(unix/ftp/vsftpd_234_backdoor) > sessions

Active sessions

Id  Name  Type      Information  Connection
--  ---  -
2   shell cmd/unix  192.168.76.128:44043 -> 192.168.76.130:6200
```



Commands - back and exit.

- `back`: exit from a module context.
- `exit`: exit from msfconsole.

```
msf6 exploit(windows/browser/ms14_064_ole_code_execution) >  
msf6 exploit(windows/browser/ms14_064_ole_code_execution) > back  
msf6 >  
msf6 > exit  
  
❏(kali@kali)-[~]  
$
```



The Main Steps of Launching Attacks.

- Search the exploits for a vuln (using those keywords related to this vuln)
- Select the exploit with a good rank using `use <exploit name>.`



The Main Steps of Launching Attacks. (continued)

- Show compatible payloads with `show payloads`.
- Select the payload using `set payload <payload name>`.



The Main Steps of Launching Attacks. (continued)

- Show exploit and payload options with `show options`.
- Set an option with `set <option name> <value>`.
- Launch the attack with `exploit`.



Notes for the main steps.

- This sequence of steps has a natural logic behind it. If you understand the logic, it is very easy for you to remember these seven steps.
- The steps 3 and 4 for selecting a payload are optional.
- If you omit steps 3 and 4, MSF will pick a suitable one according to its own intelligence.



An Example - Exploiting the UnrealIRCd vulnerability.

- According to the GVM report on Metasploitable2, it runs the Unreal IRC daemon UnrealIRCd version 3.2.8.1.
- Note: IRC (Internet Relay Chat), a chatting tool
- This version contains a backdoor, which can be triggered by sending the daemon letters "AB" followed by a system command.
- Let's see how we can exploit this vuln using MSF.



An Example - Step 1.

- Choosing a good search string is very important.
- For this example, all of the following strings will lead you to the exploit you want.

```
msf6 > search unrealircd
```

Matching Modules

#	Name	Disclosure Date	Rank
0	exploit/unix/irc/unreal_ircd_3281_backdoor	2010-06-12	excellent



An Example - Step 1. (continued)

- Unreal_ircd
- Unrealircd
- cve-2010-2075



An Example - Step 2.

- Since the returned exploit has a good rank, select it by the 'use' command.

```
msf > use exploit/unix/irc/unreal_ircd_3281_backdoor  
msf exploit(unreal_ircd_3281_backdoor) >
```



An Example - Step 2. (continued)

- You can also specify this exploit by its index from search results:
- `use 0`
- is equivalent to:
- `use exploit/unix/irc/unreal_ircd_3281_backdoor`



An Example - Step 3.

- `show payloads`
- You can use the 'info' command to get detailed description about a payload.

```
msf exploit(unreal_ircd_3281_backdoor) > show payloads

Compatible Payloads
=====
```

Name	Disclosure Date	Rank
----	-----	----
cmd/unix/bind_perl		normal
cmd/unix/bind_perl_ipv6		normal
cmd/unix/bind_ruby		normal
cmd/unix/bind_ruby_ipv6		normal
cmd/unix/generic		normal
cmd/unix/reverse		normal
cmd/unix/reverse_perl		normal
cmd/unix/reverse_perl_ssl		normal



An Example - Step 3.

- Understanding the payload name:
- 'cmd' means this payload will give you a command shell.
- 'unix' means this payload works in unix-family OSes.
- 'bind' and 'reverse' tell us about the stager and hence its direction of TCP connection.



An Example - Step 4.

- Select a payload. Suppose we select the first one, knowing that
- The connection to target will not be blocked by firewall.

```
msf exploit(unreal_ircd_3281_backdoor) > set payload cmd/unix/bind_perl  
payload => cmd/unix/bind_perl
```



An Example - Step 4. (continued)

- The perl program is available on target machine



An Example - Step 5.

- `show options`

```
Module options (exploit/unix/irc/unreal_ircd_3281_backdoor):
```

Name	Current Setting	Required	Description
RHOSTS		yes	The target host(s), range CIDR
file:<path>'			
RPORT	6667	yes	The target port (TCP)

```
Payload options (cmd/unix/bind_perl):
```

Name	Current Setting	Required	Description
LPORT	4444	yes	The listen port
RHOST		no	The target address



An Example - Step 6.

- Set the missing RHOSTS and RHOST values. Option names are case-insensitive.

```
msf5 exploit(unix/irc/unreal_ircd_3281_backdoor) > set rhosts 192.168.137.129  
rhosts => 192.168.137.129
```



An Example - Step 6. (continued)

- The setting of RHOSTS will be populated to RHOST automatically, so you don't need to set RHOST separately.



References

- Offensive Security, *Metasploit Unleashed*:
<https://www.offensive-security.com/metasploit-unleashed/>



Report Lab Work 1 & 2

- Set up the Lab 1 environment and submit the results:
- Each student chooses a pair of IP addresses.
- Example: Nguyen Van A:
- WAN: 10.10.1.2/24
- Server: 172.16.1.2/24
- Set up Nmap and OpenVAS, perform the scan, and submit the results.
- Deadline: 12 AM - Thursday, May 30, 2025



Institute of
Information Technology

Thank You for Listening!

Lab Work 02 – Security Threats and Scanning